

# Introduction to Python Programming

## Chapter 14: Data Structures - Intermediate Concepts

### Introduction

Now that you understand the basic data structures, it's time to explore how they work together. Real-world data is often complex and requires nested structures - lists within dictionaries, sets within tuples, and more. This chapter also introduces JSON (a standard format for web data) and practical applications like GPS coordinate calculations using tuples.

### 1. Nested Data Structures

Data structures can contain other data structures, creating powerful ways to organize complex information.

#### Lists Within Lists (2D Lists/Matrices)

```
# Matrix (2D list)
matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]

# Access elements
print(matrix[0])           # [1, 2, 3] (first row)
print(matrix[0][0])        # 1 (first row, first column)
print(matrix[1][2])        # 6 (second row, third column)

# Iterate through 2D list
for row in matrix:
    for item in row:
        print(item, end=" ")
    print() # New line after each row
```

#### Output:

```
1 2 3
4 5 6
7 8 9
```

**Use cases:** Spreadsheets, game boards, images (pixels), mathematical matrices

## Sets Within Sets

Wait - you can't actually put sets inside sets! Sets can only contain immutable items, and sets themselves are mutable.

```
# This DOESN'T work
# outer = {{1, 2}, {3, 4}} # ERROR!

# But you CAN use frozensets (immutable sets)
outer = {frozenset([1, 2]), frozenset([3, 4])}
print(outer) # {frozenset({1, 2}), frozenset({3, 4})}
```

**Frozensets** are immutable versions of sets - they can be elements of other sets or dictionary keys.

## Tuples Within Tuples

```
# Nested tuples
coordinates_3d = ((1, 2, 3), (4, 5, 6), (7, 8, 9))

# Access nested elements
print(coordinates_3d[0]) # (1, 2, 3)
print(coordinates_3d[0][1]) # 2

# Multiple GPS points
locations = (
    ("Calgary", (51.0447, -114.0719)),
    ("Toronto", (43.6532, -79.3832)),
    ("Vancouver", (49.2827, -123.1207))
)

# Access city and coordinates
city, coords = locations[0]
print(f"{city}: {coords}") # Calgary: (51.0447, -114.0719)
```

**Use cases:** Fixed collections of fixed data (coordinates, date-time tuples)

## Dictionaries Within Dictionaries

```
# Nested dictionaries
students = {
    "alice": {
        "age": 20,
        "grade": "A",
        "courses": ["Math", "Physics", "CS"]
    },
    "bob": {
```

```

        "age": 21,
        "grade": "B",
        "courses": ["Biology", "Chemistry"]
    }
}

# Access nested data
print(students["alice"]["age"])          # 20
print(students["alice"]["courses"][0])  # Math

# Iterate through nested dictionary
for name, info in students.items():
    print(f"{name}:")
    print(f"    Age: {info['age']}")
    print(f"    Grade: {info['grade']}")
    print(f"    Courses: {' '.join(info['courses'])}")

```

**Use cases:** Configuration files, API responses, hierarchical data

## Mixed Nesting Examples

```

# Dictionary with lists and tuples
classroom = {
    "course": "Python Programming",
    "students": ["Alice", "Bob", "Charlie"],
    "location": ("Building A", "Room 101"),
    "schedule": [
        ("Monday", "9:00 AM"),
        ("Wednesday", "9:00 AM"),
        ("Friday", "9:00 AM")
    ]
}

# List of dictionaries
inventory = [
    {"item": "apple", "quantity": 50, "price": 1.99},
    {"item": "banana", "quantity": 30, "price": 0.99},
    {"item": "orange", "quantity": 40, "price": 2.49}
]

# Access data
print(inventory[0]["item"])    # apple
print(inventory[1]["quantity"]) # 30

# Calculate total value
total_value = 0
for product in inventory:
    total_value += product["quantity"] * product["price"]

```

```
print(f"Total inventory value: ${total_value:.2f}")
```

## 2. Introduction to JSON

**JSON (JavaScript Object Notation)** is a standard format for exchanging data, especially in web applications. Python dictionaries map perfectly to JSON!

### What is JSON?

JSON looks very similar to Python dictionaries:

```
{
    "name": "Alice",
    "age": 20,
    "courses": ["Math", "Physics"],
    "is_active": true
}
```

#### Key differences from Python:

- Uses `true` / `false` (not `True` / `False`)
- Uses `null` (not `None`)
- Only double quotes for strings
- No trailing commas

## Converting Between Python and JSON

Python's `json` module handles conversion:

```
import json

# Python dictionary
student = {
    "name": "Alice",
    "age": 20,
    "courses": ["Math", "Physics", "CS"],
    "is_active": True
}

# Convert Python to JSON string (serialization)
json_string = json.dumps(student)
print(json_string)
# {"name": "Alice", "age": 20, "courses": ["Math", "Physics", "CS"], "is_active": true}
```

```
# Convert JSON string to Python (deserialization)
python_dict = json.loads(json_string)
print(python_dict)
# {'name': 'Alice', 'age': 20, 'courses': ['Math', 'Physics', 'CS'], 'is_active': True}
```

## Pretty Printing JSON

```
import json

data = {"name": "Alice", "age": 20, "courses": ["Math", "Physics"]}

# Pretty print with indentation
pretty_json = json.dumps(data, indent=4)
print(pretty_json)
```

### Output:

```
{
    "name": "Alice",
    "age": 20,
    "courses": [
        "Math",
        "Physics"
    ]
}
```

## Saving and Loading JSON Files

```
import json

# Data to save
students = [
    {"name": "Alice", "grade": 85},
    {"name": "Bob", "grade": 92},
    {"name": "Charlie", "grade": 78}
]

# Save to file
with open("students.json", "w") as file:
    json.dump(students, file, indent=4)

# Load from file
with open("students.json", "r") as file:
    loaded_data = json.load(file)
    print(loaded_data)
```

# Why JSON Matters

## JSON is everywhere:

- Web APIs return data in JSON format
- Configuration files often use JSON
- Data exchange between different programming languages
- Storing structured data in files

**Python dictionaries make working with JSON natural** because the structures match so well!

## Practical JSON Example

```
import json

# API-like response structure
api_response = {
    "status": "success",
    "data": {
        "user": {
            "id": 12345,
            "name": "Alice",
            "email": "alice@example.com"
        },
        "posts": [
            {"id": 1, "title": "First Post", "likes": 10},
            {"id": 2, "title": "Second Post", "likes": 25}
        ]
    }
}

# Convert to JSON
json_data = json.dumps(api_response, indent=2)
print(json_data)

# Access nested data after loading
user_name = api_response["data"]["user"]["name"]
first_post = api_response["data"]["posts"][0]["title"]
print(f"User: {user_name}, First Post: {first_post}")
```

---

## 3. Tuples for GPS Coordinates

Tuples are perfect for GPS coordinates because:

- Latitude and longitude always go together

- They shouldn't be modified independently
- Immutable nature prevents accidental changes

## Representing GPS Locations

```
# Format: (latitude, longitude)
calgary = (51.0447, -114.0719)
toronto = (43.6532, -79.3832)
vancouver = (49.2827, -123.1207)

# Unpack coordinates
lat, lon = calgary
print(f"Calgary is at {lat}°N, {lon}°W")
```

## Calculating Distance Between Coordinates

Using the **Haversine formula** to calculate distance between two GPS points:

```
import math

def haversine_distance(coord1, coord2):
    """
    Calculate distance between two GPS coordinates in kilometers.
    coord1, coord2: tuples of (latitude, longitude) in decimal degrees
    """
    # Unpack coordinates
    lat1, lon1 = coord1
    lat2, lon2 = coord2

    # Earth's radius in kilometers
    R = 6371

    # Convert degrees to radians
    lat1_rad = math.radians(lat1)
    lat2_rad = math.radians(lat2)
    delta_lat = math.radians(lat2 - lat1)
    delta_lon = math.radians(lon2 - lon1)

    # Haversine formula
    a = math.sin(delta_lat/2)**2 + math.cos(lat1_rad) * math.cos(lat2_rad) *
math.sin(delta_lon/2)**2
    c = 2 * math.asin(math.sqrt(a))

    distance = R * c
    return distance
```

```
# Calculate distances
calgary = (51.0447, -114.0719)
toronto = (43.6532, -79.3832)
vancouver = (49.2827, -123.1207)

distance_cal_tor = haversine_distance(calgary, toronto)
distance_cal_van = haversine_distance(calgary, vancouver)

print(f"Calgary to Toronto: {distance_cal_tor:.2f} km")
print(f"Calgary to Vancouver: {distance_cal_van:.2f} km")
```

## Finding a Point North/South/East/West

Calculate new GPS coordinates at a given distance and direction:

```
import math

def point_at_distance(start_coord, distance_km, bearing_degrees):
    """
    Calculate GPS coordinates at a given distance and direction.
    start_coord: tuple (latitude, longitude)
    distance_km: distance in kilometers
    bearing_degrees: 0=North, 90=East, 180=South, 270=West
    Returns: tuple (new_latitude, new_longitude)
    """
    lat1, lon1 = start_coord
    R = 6371 # Earth's radius in km

    # Convert to radians
    lat1_rad = math.radians(lat1)
    lon1_rad = math.radians(lon1)
    bearing_rad = math.radians(bearing_degrees)

    # Calculate new latitude
    lat2_rad = math.asin(
        math.sin(lat1_rad) * math.cos(distance_km/R) +
        math.cos(lat1_rad) * math.sin(distance_km/R) * math.cos(bearing_rad)
    )

    # Calculate new longitude
    lon2_rad = lon1_rad + math.atan2(
        math.sin(bearing_rad) * math.sin(distance_km/R) * math.cos(lat1_rad),
        math.cos(distance_km/R) - math.sin(lat1_rad) * math.sin(lat2_rad)
    )

    # Convert back to degrees
    lat2 = math.degrees(lat2_rad)
    lon2 = math.degrees(lon2_rad)
```



```

    return (lat2, lon2)

# Example usage
calgary = (51.0447, -114.0719)

# Find point 5km north of Calgary
north_5km = point_at_distance(calgary, 5, 0)
print(f"5km North of Calgary: {north_5km}")

# Find point 10km east of Calgary
east_10km = point_at_distance(calgary, 10, 90)
print(f"10km East of Calgary: {east_10km}")

# Find point 8km south of Calgary
south_8km = point_at_distance(calgary, 8, 180)
print(f"8km South of Calgary: {south_8km}")

```

## GPS Location System

```

def create_location(name, lat, lon):
    """Create a location tuple with name and coordinates."""
    return (name, (lat, lon))

def display_location(location):
    """Display location information."""
    name, (lat, lon) = location
    print(f"{name}: {lat}°, {lon}°")

def distance_between(loc1, loc2):
    """Calculate distance between two locations."""
    _, coord1 = loc1
    _, coord2 = loc2
    return haversine_distance(coord1, coord2)

# Create locations
calgary = create_location("Calgary", 51.0447, -114.0719)
banff = create_location("Banff", 51.1784, -115.5708)

# Display
display_location(calgary)
display_location(banff)

# Calculate distance
dist = distance_between(calgary, banff)
print(f"Distance: {dist:.2f} km")

```

## 4. Practical Applications

### Example 1: Student Management System

```
# Nested structure: dictionary of dictionaries with lists
classroom = {
    "Python101": {
        "instructor": "Dr. Smith",
        "students": [
            {"name": "Alice", "id": "001", "grades": [85, 90, 92]},
            {"name": "Bob", "id": "002", "grades": [78, 82, 88]},
            {"name": "Charlie", "id": "003", "grades": [95, 93, 97]}
        ],
        "schedule": [
            ("Monday", "9:00 AM", "Room 101"),
            ("Wednesday", "9:00 AM", "Room 101"),
            ("Friday", "9:00 AM", "Room 101")
        ]
    }
}

# Access and calculate
course = classroom["Python101"]
print(f"Instructor: {course['instructor']}")

for student in course["students"]:
    avg = sum(student["grades"]) / len(student["grades"])
    print(f"{student['name']} (ID: {student['id']}): Average = {avg:.2f}")
```

### Example 2: GPS Trip Planner

```
# List of location tuples
trip = [
    ("Start: Calgary", (51.0447, -114.0719)),
    ("Stop 1: Banff", (51.1784, -115.5708)),
    ("Stop 2: Lake Louise", (51.4254, -116.1773)),
    ("End: Jasper", (52.8737, -118.0814))
]

# Calculate total trip distance
total_distance = 0

for i in range(len(trip) - 1):
    name1, coord1 = trip[i]
    name2, coord2 = trip[i + 1]
```

```

distance = haversine_distance(coord1, coord2)
total_distance += distance

print(f"{name1} to {name2}: {distance:.2f} km")

print(f"\nTotal trip distance: {total_distance:.2f} km")

```

## Example 3: Data Export to JSON

```

import json

# Complex Python structure
company_data = {
    "company": "Tech Solutions Inc.",
    "employees": [
        {
            "id": 1,
            "name": "Alice Johnson",
            "position": "Developer",
            "skills": ["Python", "JavaScript", "SQL"],
            "location": (51.0447, -114.0719) # Tuple
        },
        {
            "id": 2,
            "name": "Bob Smith",
            "position": "Designer",
            "skills": ["Photoshop", "Illustrator"],
            "location": (43.6532, -79.3832)
        }
    ]
}

# Note: JSON can't handle tuples directly, they become lists
json_output = json.dumps(company_data, indent=2)
print(json_output)

# When loading back, tuples become lists
loaded_data = json.loads(json_output)
print(type(loaded_data["employees"][0]["location"])) # <class 'list'>

# Convert back to tuple if needed
for employee in loaded_data["employees"]:
    employee["location"] = tuple(employee["location"])

```

---

## 5. Common Patterns and Best Practices

## Pattern 1: Nested Dictionary Access with get()

```
data = {
    "user": {
        "profile": {
            "name": "Alice"
        }
    }
}

# Unsafe - can cause KeyError
# name = data["user"]["profile"]["name"]

# Safe - returns None if key missing
name = data.get("user", {}).get("profile", {}).get("name")
print(name) # Alice
```

## Pattern 2: Building Nested Structures

```
# Start empty, build incrementally
students = {}

def add_student(name, course, grade):
    if name not in students:
        students[name] = {"courses": {}}
    students[name]["courses"][course] = grade

add_student("Alice", "Math", 85)
add_student("Alice", "Physics", 90)
add_student("Bob", "Math", 92)

print(students)
# {'Alice': {'courses': {'Math': 85, 'Physics': 90}},
#  'Bob': {'courses': {'Math': 92}}}
```

## Pattern 3: Flattening Nested Lists

```
nested = [[1, 2, 3], [4, 5], [6, 7, 8, 9]]

# Flatten to single list
flattened = []
for sublist in nested:
    for item in sublist:
        flattened.append(item)

print(flattened) # [1, 2, 3, 4, 5, 6, 7, 8, 9]

# Or using list comprehension (more advanced)
flattened = [item for sublist in nested for item in sublist]
```

---

## Key Takeaways

- ✓ **Nested structures** allow complex data organization (lists in dicts, dicts in lists, etc.)
- ✓ **JSON** is a standard format for data exchange, maps naturally to Python dictionaries
- ✓ **json.dumps()** converts Python to JSON (serialization), **json.loads()** converts back
- ✓ **Tuples are perfect for coordinates** - immutable paired data like (lat, lon)
- ✓ **Haversine formula** calculates distance between GPS coordinates
- ✓ **Bearing calculations** find new points at given distance and direction
- ✓ **Nested dictionaries** commonly used for configuration, API responses, hierarchical data
- ✓ **Mixed nesting** combines structures for flexibility (list of dicts, dicts with tuples, etc.)
- ✓ **Safe access** with `.get()` prevents `KeyError` in nested structures
- ✓ **JSON limitations** - tuples become lists, need conversion when loading

---

## Reflection Questions

1. Why are tuples better than lists for GPS coordinates?
2. What's the difference between `json.dumps()` and `json.dump()`?
3. How do you safely access deeply nested dictionary values?
4. Why can't you put a set inside another set?
5. What happens to Python tuples when converted to JSON?
6. What are bearings in GPS calculations? (0°, 90°, 180°, 270°)
7. When would you use a list of dictionaries vs a dictionary of lists?

8. How is JSON used in web applications?

---

## Practice Exercises

---

1. **2D Grid:** Create a 5x5 matrix, fill with numbers 1-25, print in grid format
2. **Nested Dict:** Create dictionary of dictionaries for 3 students with name, age, grades
3. **JSON Export:** Convert a Python dictionary to JSON, save to file, load it back
4. **GPS Calculator:** Calculate distance between 3 cities using their coordinates
5. **Find North:** Given coordinates, find point 10km directly north
6. **Trip Distance:** List of 5 GPS waypoints, calculate total trip distance
7. **Data Structure:** Create list of dicts for products (name, price, quantity), calculate total value
8. **API Response:** Parse nested JSON structure to extract specific values
9. **Location Finder:** Function that takes city name, returns coordinates from nested dict
10. **Matrix Operations:** Create 3x3 matrix, write function to sum each row and column